



# Distributed Information Processing

6<sup>th</sup> Lecture

Eom, Hyeonsang (엄현상)  
Department of Computer Science  
& Engineering  
Seoul National University



# Outline

- Middleware Communication Services
  - Remote Procedure Call
  - Message-Oriented Communication
    - Message-oriented transient communication
    - Message-oriented persistent communication
  - Stream-Oriented Communication
- Q&A

# Conventional Procedure Call

Figure 4-5 on Page 127

*(a) Parameter passing in a local procedure call: the stack before the call to read. (b) The stack while the called procedure is active*

# Client and Server Stubs

Figure 4-6 on Page 128

*Principle of RPC between a client and server program*

Client Stub: packs the parameters into a message and requests it to be sent  
Server Stub: transforms the incoming requests into local procedure calls

# Remote Procedure Calls (RPCs)

## ■ RPC Occurring in the Following Steps:

1. The client procedure calls the client stub in the normal way.
2. The client stub builds a message and calls the local operating system.
3. The client's OS sends the message to the remote OS.
4. The remote OS gives the message to the server stub.
5. The server stub unpacks the parameters and calls the server.

## RPCs (Cont'd)

- RPC Occurring in the Following Steps:
  6. The server does the work and returns the result to the stub.
  7. The server stub packs it in a message and calls its local OS.
  8. The server's OS sends the message to the client's OS.
  9. The client's OS gives the message to the client stub.
  10. The stub unpacks the result and returns to the client.



# Passing Value Parameters

Figure 4–7 on Page 130

*The steps involved in doing a remote computation through RPC*

# Passing Value Parameters (Cont'd)

- Illustration: Little Endian Format
  - Numbering Bytes Right to Left

Figure 4-8 (a) on Page 131

*The original message on the Pentium*

# Passing Value Parameters (Cont'd)

- Illustration: Big Endian Format
  - Numbering Bytes Left to Right

Figure 4-8 (b) on Page 131

*The message after receipt on the SPARC*

# Passing Value Parameters (Cont'd)

## ■ Illustration: Big Endian Format

Figure 4-8 (c) on Page 131

*The message after being inverted*

*The little numbers in boxes indicate the address of each byte*

Need for additional information about what is a string and what is an integer!

# Parameter Specification and Stub Generation

Figure 4-9 (a) on Page 133

Figure 4-9 (b) on Page 133

*(a) A procedure*

*(b) The corresponding message*

# Asynchronous RPC

Figure 4–10 (a) on Page 135

The interaction between client and server in a traditional RPC

# Asynchronous RPC (Cont'd)

Figure 4–10 (b) on Page 135

The interaction using *asynchronous* RPC

# Asynchronous RPC (Cont'd)

Figure 4-11 on Page 135

*A client and server interacting through two asynchronous RPCs*

# Example: DCE RPC

- DCE (Distributed Computing Environment)
  - Middleware System Designed to Execute as a Layer of Abstraction between Existing (Network) OSes and Distributed Applications
    - All communication between clients and servers takes place by means of RPC
  - One Providing Services
    - Distributed File Services
    - Directory Services
    - Security Services
    - Distributed Time Services
  - One Developed by Open Software Foundation Called Now Open Group



# Writing a Client and a Server

Figure 4–12 on Page 138

*The steps in writing a client and a server in DCE RPC*

# Writing a Client and a Server (Cont'd)

- Three Files Output by the IDL  
(Interface Definition Language)  
Compiler
  - A header file (e.g., interface.h, in C terms)
  - The client stub
  - The server stub



# Binding a Client to a Server

- Registration of a Server Makes It Possible for a Client to Locate the Server and Bind to It
- Server Location Is Done in Two Steps
  1. Locate the server's machine
  2. Locate the server on that machine



# Binding a Client to a Server (Con'td)

Figure 4-13 on Page 140

Client-to-Server Binding in DCE



# Berkeley Sockets

Figure 4-14 on Page 142

The socket primitives for TCP/IP



# Berkeley Sockets (Cont'd)

Figure 4-15 on Page 143

Connection-oriented communication pattern using sockets



# Message-Passing Interface (MPI)

Figure 4-16 on Page 144

*Some of the most intuitive message-passing primitives of MPI*



# Message-Queuing Model

Figure 4-17 on Page 146

*Four combinations for loosely-coupled communications using queues*



# General Architecture of a Message-Queuing System

Figure 4-19 on Page 148

*The relationship between queue-level addressing and network-level addressing*



# General Architecture of a Message-Queuing System (Cont'd)

Figure 4-20 on Page 149

**General organization of a message-queuing system with routers**

# Message Brokers

Application-level gateway  
converting the format

Message broker

Figure 4-21 on Page 150

*The general organization of a message broker in a message-queuing system*



# Data Stream

Figure 4–26 on Page 160

*A general architecture for streaming stored multimedia data over a network*

# Streams and Quality of Service

- Properties for Quality of Service
  - The required bit rate at which data should be transported
  - The maximum delay until a session has been set up
  - The maximum end-to-end delay
  - The maximum delay variance, or jitter
  - The maximum round-trip delay



# Enforcing QoS

Figure 4–27 on Page 161

*Using a buffer to reduce jitter*

# Enforcing QoS (Cont'd)

Figure 4-28 on Page 162

The effect of packet loss in (a) non interleaved transmission  
and (b) interleaved transmission



# Synchronization Mechanisms

Figure 4–29 on Page 164

*The principle of explicit synchronization on the data units of simple streams*

# Synchronization Mechanisms (Cont'd)

Figure 4–30 on Page 165

*The principle of synchronization as supported by high-level interfaces*